

A* Grid Navigation Agent

An autonomous problem-solving agent that computes the shortest obstacle-free path across a 2D grid using A* informed search — implemented in TypeScript and SvelteKit, executing entirely in the browser with no backend, database or external API.

Course Code	[COURSE CODE]	Group ID	[GROUP ID]
Team Members	[MEMBER 1], [MEMBER 2], [MEMBER 3]	Selected Track	A* Search / Problem-Solving Agent
Project Name	A* Grid Navigation Agent	Tech Stack	TypeScript, SvelteKit (client-side only)
Repository	https://github.com/voltcrash/automaze	Live Demo	https://voltcrash.com/automaze

1 PEAS FRAMEWORK

COMPONENT	SPECIFICATION
Performance Measure	Reach the goal successfully; minimise total path cost; find the shortest valid route; reduce unnecessary node exploration.
Environment	A 2D grid comprising a start cell, a goal cell, free traversable cells, and obstacle/wall cells. Fully observable, static, discrete and deterministic.
Actuators	Move Up; Move Down; Move Left; Move Right.
Sensors	Current agent position; valid neighbouring cells; obstacle locations; goal location.

2 CORE ALGORITHMIC FORMULATION

State Space	Each valid grid cell (x , y) represents one possible state.
Initial State	The agent's starting grid position.
Goal Test	The current position equals the goal position.
Actions	Move Up, Down, Left or Right into a valid, in-bounds, non-obstacle cell.
Path Cost	Each movement costs 1 ; the cost of a path is its number of moves.

$$f(n) = g(n) + h(n)$$

$g(n)$ — actual cost from the start to node n · $h(n)$ — estimated cost from n to the goal · $f(n)$ — estimated total path cost through n

$$h(n) = |x_{goal} - x_n| + |y_{goal} - y_n|$$

Manhattan Distance

Manhattan distance is the appropriate heuristic because movement is restricted to four directions, so no diagonal shortcut exists. It therefore never overestimates the true remaining cost — it is **admissible and consistent**, which guarantees that A* returns an **optimal** path. The agent expands the open-list node with the lowest $f(n)$, breaking ties on the lower $h(n)$.

3 COMPLEXITY & OBSERVED PERFORMANCE

THEORETICAL COMPLEXITY	BOUND	OBSERVED METRIC	RESULT
Time (worst case)	$O(b^d)$, $b \leq 4$	Final Path Cost	[VALUE]
Time (finite grid)	$O(N \log N)$ with a heap	Final Path Length	[VALUE]
Space	$O(N)$ — open + closed lists	Nodes Expanded	[VALUE]
		Visited Nodes	[VALUE]
		Execution Time	[VALUE] ms

Here b is the branching factor (at most 4), d the depth of the optimal solution, and N the number of traversable cells. On a finite grid each cell is expanded at most once, so the practical cost is bounded by the grid size rather than by b^d . Actual performance depends on **the number of states explored** and **the quality of the heuristic**: a sharper admissible $h(n)$ expands fewer nodes for the same optimal path. The application measures the metrics above at runtime and reports them after each search.